1. INTRODUCTION

1.1 Introduction

The rapid expansion of e-commerce has revolutionized shopping habits, allowing consumers to purchase products and services online conveniently. However, as online shopping grows, so do the challenges consumers face in navigating an overwhelming number of options, ensuring they receive the best prices, and identifying credible product reviews. These issues create friction in the shopping experience, often leaving consumers unsure about whether they have received the best value and reliable information.

In today's competitive e-commerce landscape, many consumers search across multiple platforms to find the best deals, but manual price comparison can be time-consuming and inefficient. Furthermore, even after finding a product at an ideal price, prices frequently fluctuate due to promotions, seasonal sales, and other market factors, making it difficult for users to capitalize on the best moments to purchase. A real-time price alert system could empower consumers by notifying them of price drops, enabling timely decision-making.

Additionally, fake reviews have become a major concern in e-commerce. Studies reveal that fraudulent reviews manipulate ratings, skew perceptions of product quality, and mislead buyers. Many vendors exploit this vulnerability by inflating positive reviews or creating negative reviews for competitors, leading to a deceptive marketplace. This erosion of trust impacts consumers' confidence in online reviews, a crucial resource for informed decision-making.

This project addresses these challenges by developing an integrated e-commerce platform that combines three critical features:

- Price Comparison – Aggregates pricing and availability from multiple websites, allowing users to compare product prices in real-time.
- Price Alerts – Provides customized notifications for price drops based on user-defined thresholds.
- Fake Review Detection – Implements machine learning techniques to detect potentially fraudulent reviews, presenting only authentic reviews to users.

By offering these features, the platform aims to create a seamless, transparent, and trustworthy shopping experience. It enables consumers to make informed decisions confidently, find the best deals, and access reliable feedback on products. In doing so, this platform seeks to enhance user satisfaction, trust, and engagement in online shopping.

1.2 Problem statement

The impact of online reviews on businesses has grown significantly during last years, being crucial to determine business success in a wide array of sectors, ranging from restaurants, hotels to e-commerce. Unfortunately, some users use unethical means to improve their online reputation by writing fake reviews of their businesses or competitors. Previous research has addressed fake review detection in a number of domains, such as product or business reviews in restaurants and hotels.

In this paper, we make some classification approaches for detecting fake online reviews some of which are semi supervised and others are supervised. For semi-supervised learning, we use label Propagation and label spreading algorithms. K Nearest Neighbors , Random Forest and Multi layers algorithms n are used as classifiers in our research work to improve the performance of classification. We have mainly focused on the content of the review based approaches.

1.3 Motivation

Content based methods focus on what is the content of the review. That is the text of the review or what is told in it. It have attempted to detect spam review by analyzing the linguistic features of the review. It used three techniques to perform classification. These three techniques are- genre identification, detection of psycholinguistic deception and text categorization.

Behavior feature based study focuses on the reviewer that includes characteristics of the person who is giving the review. It addressed the problem of review spammer detection, or finding users who are the source of spam reviews. People who post intentional fake reviews have significantly different behavior than the normal user. They have identified the following deceptive rating and review behaviors. Deceptive online review detection is generally considered as a classification problem and one popular approach is to use supervised text classification techniques [5]. These techniques are robust if the training is

performed using large datasets of labeled instances from both classes, deceptive opinions (positive instances) and truthful opinions (negative examples) [8]. Some researchers also used semi-supervised classification techniques.

1.4 Objective

In this paper, we make some classification approaches for detecting fake online reviews some of which are semi supervised and others are supervised. For semi-supervised learning, we use label Propagation and label spreading algorithms. K Nearest Neighbors , Random Forest and Multi layers algorithms n are used as classifiers in our research work to improve the performance of classification. We have mainly focused on the content of the review based approaches.

1.5 Existing System

Content based methods focus on what is the content of the review. That is the text of the review or what is told in it.we have attempted to detect spam review by analyzing the linguistic features of the review.It used three techniques to perform classification. These three techniques are- genre identification, detection of psycholinguistic deception and text categorization. Behaviour feature based study focuses on the reviewer that includes characteristics of the person who is giving the review.

Disadvantages Of Existing System

➤ In the existing work, the system uses only to semi-supervised learning. Only Text Classification as sentiment text and it never finds fake.

1.6 Proposed System

In the proposed system, each review goes through tokenization process first. Then unnecessary words are removed and candidate feature words are generated.

Each candidate feature words are checked against the dictionary and if its entry is available in the dictionary then its frequency is counted and added to the column in the feature vector that corresponds the numeric map of the word.A longside with counting frequency, the length of the review is measured and added to the feature vector. Finally, sentiment score which is available in the data set is added in the feature vector. We have

assigned negative sentiment as zero valued and positive sentiment as some positive valued in the feature vector.

Advantages of proposed system :

The system is very fast and effective due to semi-supervised and supervised learning. Focused on the content of the review based approaches. As feature we have used word frequency count, sentiment polarity and length of review.

2. REQUIREMENT ANALYSIS

2.1 Feasibility Study

A feasibility study evaluates whether the project is practical, achievable, and beneficial in its current scope. For this project:

The project aims to develop a system for price comparison, price alerts, and fake review detection in e-commerce, addressing significant challenges like consumer trust and decision-making. Incorporates modern machine learning techniques and data mining algorithms, which are proven to be effective in similar use cases. The feasibility assessment involves analyzing technical, economic, and social factors.

2.1.1 Technical Feasibility

- Uses proven machine learning algorithms like Random Forest and Naïve Bayes.
- Implements Python libraries such as TensorFlow, Pandas, and Scikit-learn.
- Employs tag path analysis and Lucene for data crawling and extraction.
- Requires moderate hardware (standard processors and basic storage).
- Relies on open-source tools, making it easy and cost-effective to deploy.

2.1.2 Economical Feasibility

- **Low Costs:** Utilizes open-source tools like Python, TensorFlow, and Lucene, reducing software expenses.
- **Minimal Infrastructure:** Requires moderate hardware and scalable cloud resources if needed.
- **High ROI:** Improves consumer trust and e-commerce reliability, attracting more users.
- **Cost Savings:** Reduces losses from fake reviews and phishing, benefiting businesses and consumers.
- **Viability:** Economically feasible due to low setup costs and potential for long-term financial benefits.

2.1.3 Social Feasibility

- **Consumer Trust:** Addresses issues like fake reviews and phishing, enhancing confidence in e-commerce.
- **User Acceptance:** Simplifies decision-making through price comparison and review authenticity.
- **Ethical Approach:** Promotes transparency without violating user privacy.
- **Positive Impact:** Encourages ethical business practices and improves the online shopping experience.
- **Adoption:** Socially feasible as it aligns with consumer expectations and digital trends.

2.2 Software Requirements Specifications

2.2.1 Hardware Requirement

Processor	:	Pentium –III
Speed	:	2.4 GHz
RAM	:	512 MB (min)
Hard Disk	:	20 GB
Floppy Drive	:	1.44 MB
Key Board	:	Standard Keyboard
Monitor	:	15 VGA Colour

2.2.2 Software Requirement

Operating System	:	Windows
Coding Language	:	Python 3.7

3. TECHNOLOGY USED

3.1 What is Python

Below are some facts about Python. Python is currently the most widely used multi-purpose, high-level programming language. Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following-

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like OpenCV, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

- Extensive Libraries
- Extensible
- Embeddable
- Improved Productivity
- IOT Opportunities
- Simple and Easy
- Readable

Disadvantages of Python

- Speed Limitations
- Weak in Mobile Computing and Browsers
- Design Restrictions
- Underdeveloped Database Access
- Simple

3.2 Modules Used in Project

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used

for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem.

Matplotlib

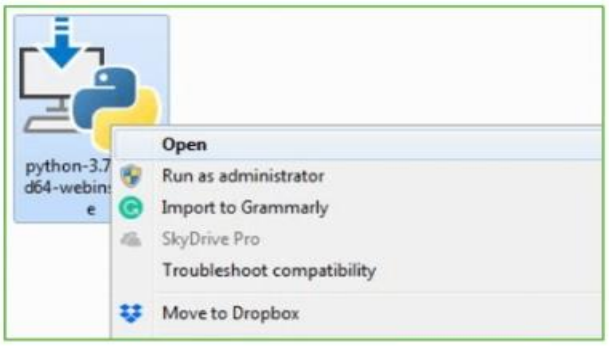
Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Installation of Python

- Go to Download and Open the downloaded python version to carry out the installation process.



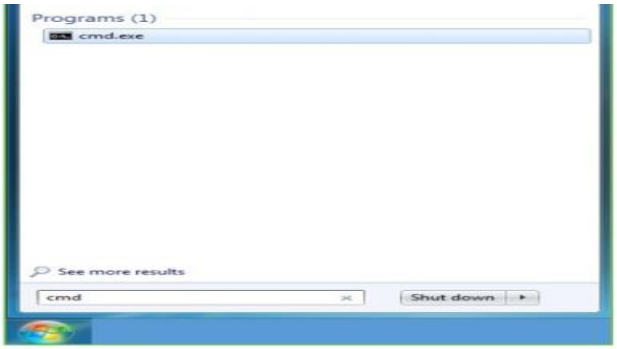
- Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

- Click on Install NOW After the installation is successful. Click on Close.

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

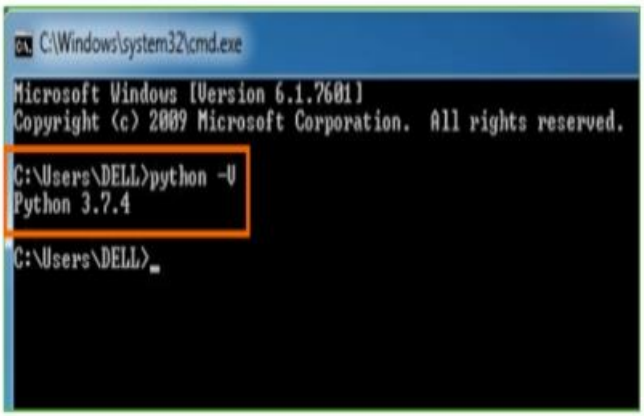
Verify the Python Installation

- Click on Start
- In the Windows Run Command, type “cmd”



- Open the Command prompt option.

- Let us test whether the python is correctly installed. Type **python –V** and press Enter.



4. ARCHITECTURE DESIGN

4.1 System Architecture

The system architecture involves the following components :

Crawling Module: Extracts data from e-commerce websites, including product records, prices, and reviews.

Data Preprocessing: Performs tokenization, removes unnecessary words, and converts reviews into numerical representations (e.g., word frequency, sentiment polarity).

Feature Extraction: Uses statistical and text-based features like review length, word frequency, and sentiment score to create feature vectors for analysis.

Machine Learning Classifier: Implements supervised (e.g., Random Forest, KNN) and semi-supervised algorithms (e.g., Label Propagation) for detecting fake reviews and phishing.

Comparison System: Matches prices across various platforms using tools like Lucene for indexing and retrieval.

Output Module: Displays price comparison, review authenticity, and alerts for suspicious websites or fake reviews.

4.2 Module Description

Crawling Module:

- Collects product and review data from multi-level website pages (homepage to subpages).
- Uses tag path analysis for structured data retrieval.

Preprocessing Module:

- Cleans and processes the extracted data.
- Converts text into numerical formats (e.g., frequency counts, sentiment scoring).

Feature Selection Module:

Reduces dimensionality by selecting relevant features like review sentiment, length, and polarity.

Fake Review Detection Module:

- Uses algorithms like Random Forest and Naïve Bayes to classify reviews as genuine or fake.
- Semi-supervised methods like Label Propagation handle limited labeled data.

Price Comparison Module:

- Crawls and indexes product prices using Lucene-based techniques.
- Compares prices across platforms to recommend the best deals.

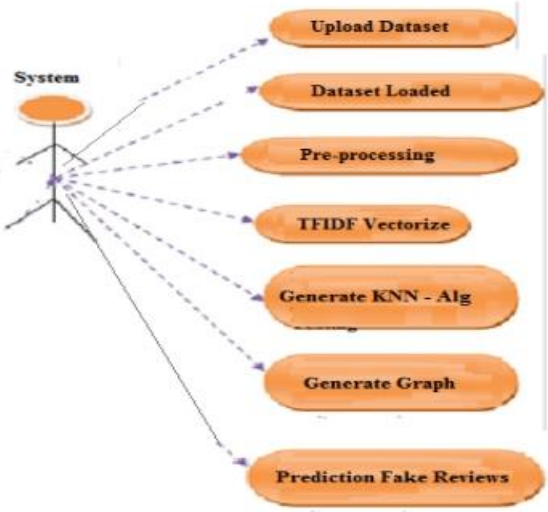
Output/Alert Module:

Displays results (e.g., authentic reviews, price comparisons) and generates alerts for phishing or suspicious activity.

4.3 UML Diagrams

Use Case Diagram

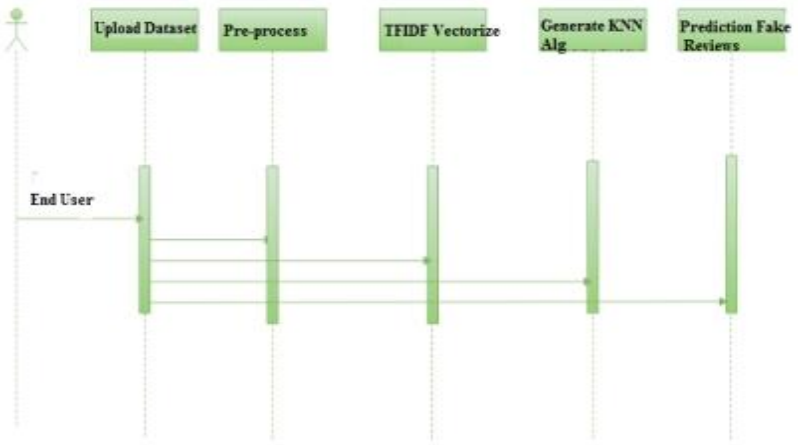
A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis.



4.3.1.1 Use Case Diagram

Sequence Diagram

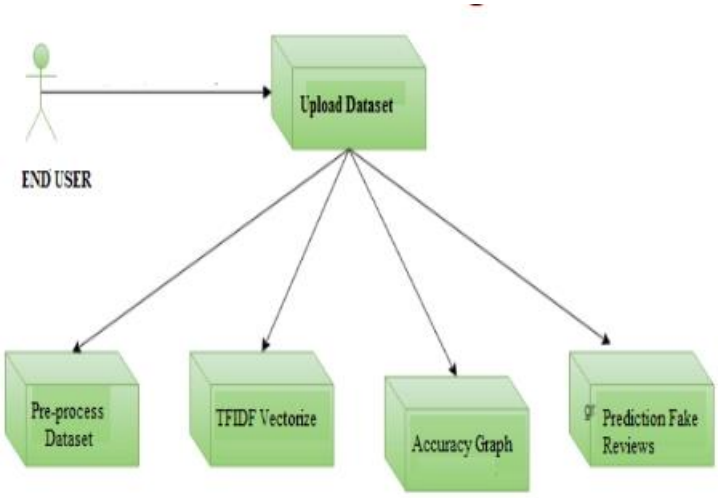
A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order.



4.31.2 Sequence Diagram

Component Diagram

It does not describe the functionality of the system but it describes the components used to make those functionalities.Thus from that point of view, component diagrams are used to visualize the physical components in a system. These components are libraries, packages, files, etc.



4.3.1.2 Component Diagram

5. IMPLEMENTATION

5.1 Sample Code

```
from __future__ import absolute_import

from __future__ import division

from __future__ import print_function

import argparse

import collections

from datetime import datetime

import hashlib

import os.path

import random

import re

import sys

import tarfile

    import numpy as np

from six.moves import urllib

import tensorflow as tf

   from tensorflow.python.framework import graph_util

from tensorflow.python.framework import tensor_shape

from tensorflow.python.platform import gfile

from tensorflow.python.util import compat

FLAGS = None

MAX_NUM_IMAGES_PER_CLASS = 2 ** 27 - 1  # ~134M

def create_image_lists(image_dir, testing_percentage, validation_percentage):
```

```
  if not gfile.Exists(image_dir):
    tf.logging.error("Image directory '" + image_dir + "' not found.")
    return None
  result = collections.OrderedDict()
  sub_dirs = [
    os.path.join(image_dir,item)
    for item in gfile.ListDirectory(image_dir)]
  sub_dirs = sorted(item for item in sub_dirs
                    if gfile.IsDirectory(item))
  for sub_dir in sub_dirs:
    extensions = ['jpg', 'jpeg', 'JPG', 'JPEG']
    file_list = []
    dir_name = os.path.basename(sub_dir)
    if dir_name == image_dir:
      continue
tf.logging.info("Looking for images in '" + dir_name + "'")
    for extension in extensions:
      file_glob = os.path.join(image_dir, dir_name, '*.' + extension)
      file_list.extend(gfile.Glob(file_glob))
    if not file_list:
      tf.logging.warning('No files found')
      continue
    if len(file_list) < 20:
      tf.logging.warning(
          'WARNING: Folder has less than 20 images, which may cause issues.')
```

```
elif len(file_list) > MAX_NUM_IMAGES_PER_CLASS:
  tf.logging.warning(
      'WARNING: Folder {} has more than {} images. Some images will '
      'never be selected.'.format(dir_name, MAX_NUM_IMAGES_PER_CLASS))
label_name = re.sub(r'[^a-z0-9]+', ' ', dir_name.lower())
training_images = []
testing_images = []
validation_images = []
for file_name in file_list:
  base_name = os.path.basename(file_name)
  hash_name = re.sub(r'_nohash_.*$', '', file_name)
  hash_name_hashed = hashlib.sha1(compat.as_bytes(hash_name)).hexdigest()
  percentage_hash = ((int(hash_name_hashed, 16) %
                      (MAX_NUM_IMAGES_PER_CLASS + 1)) *
                     (100.0 / MAX_NUM_IMAGES_PER_CLASS))
  if percentage_hash < validation_percentage:
    validation_images.append(base_name)
  elif percentage_hash < (testing_percentage + validation_percentage):
    testing_images.append(base_name)
  else:
    training_images.append(base_name)
result[label_name] = {
    'dir': dir_name,
    'training': training_images,
    'testing': testing_images,
```

```
      'validation': validation_images,
    }
  return result
def get_image_path(image_lists, label_name, index, image_dir, category):
  if label_name not in image_lists:
    tf.logging.fatal('Label does not exist %s.', label_name)
  label_lists = image_lists[label_name]
  if category not in label_lists:
    tf.logging.fatal('Category does not exist %s.', category)
  category_list = label_lists[category]
  if not category_list:
    tf.logging.fatal('Label %s has no images in the category %s.',
                     label_name, category)
  mod_index = index % len(category_list)
  base_name = category_list[mod_index]
  sub_dir = label_lists['dir']
  full_path = os.path.join(image_dir, sub_dir, base_name)
  return full_path
      def get_bottleneck_path(image_lists, label_name, index, bottleneck_dir,
                      category, architecture):
  return get_image_path(image_lists, label_name, index, bottleneck_dir,
                      category) + '_' + architecture + '.txt'
      def create_model_graph(model_info):
  with tf.Graph().as_default() as graph:
    model_path = os.path.join(FLAGS.model_dir, model_info['model_file_name'])
```

```
  with gfile.FastGFile(model_path, 'rb') as f:
    graph_def = tf.GraphDef()
    graph_def.ParseFromString(f.read())
    bottleneck_tensor, resized_input_tensor = (tf.import_graph_def(
        graph_def,
        name='',
        return_elements=[
            model_info['bottleneck_tensor_name'],
            model_info['resized_input_tensor_name'],
        ]))
  return graph, bottleneck_tensor, resized_input_tensor
      def run_bottleneck_on_image(sess, image_data, image_data_tensor,
                  decoded_image_tensor, resized_input_tensor,
                  bottleneck_tensor):
  resized_input_values = sess.run(decoded_image_tensor,
                      {image_data_tensor: image_data})
  bottleneck_values = sess.run(bottleneck_tensor,
                    {resized_input_tensor: resized_input_values})
  bottleneck_values = np.squeeze(bottleneck_values)
  return bottleneck_values
def maybe_download_and_extract(data_url):
  dest_directory = FLAGS.model_dir
  if not os.path.exists(dest_directory):
    os.makedirs(dest_directory)
  filename = data_url.split('/')[-1]
```

```
filepath = os.path.join(dest_directory, filename)
if not os.path.exists(filepath):
  def _progress(count, block_size, total_size):
    sys.stdout.write('\r>> Downloading %s %.1f%%' %
                     (filename,
                      float(count * block_size) / float(total_size) * 100.0))
    sys.stdout.flush()
        filepath, _ = urllib.request.urlretrieve(data_url, filepath, _progress)
  print()
  statinfo = os.stat(filepath)
  tf.logging.info('Successfully downloaded', filename, statinfo.st_size,
                  'bytes.')
tarfile.open(filepath, 'r:gz').extractall(dest_directory)
      def ensure_dir_exists(dir_name):
if not os.path.exists(dir_name):
  os.makedirs(dir_name)
   bottleneck_path_2_bottleneck_values = {}
    def create_bottleneck_file(bottleneck_path, image_lists, label_name, index,
                       image_dir, category, sess, jpeg_data_tensor,
                       decoded_image_tensor, resized_input_tensor,
                       bottleneck_tensor):
tf.logging.info('Creating bottleneck at ' + bottleneck_path)
image_path = get_image_path(image_lists, label_name, index,
                         image_dir, category)
if not gfile.Exists(image_path):
```

```
  tf.logging.fatal('File does not exist %s', image_path)
image_data = gfile.FastGFile(image_path, 'rb').read()
try:
  bottleneck_values = run_bottleneck_on_image(
      sess, image_data, jpeg_data_tensor, decoded_image_tensor,
      resized_input_tensor, bottleneck_tensor)
except Exception as e:
  raise RuntimeError('Error during processing file %s (%s)' % (image_path,
                                                               str(e)))
bottleneck_string = ','.join(str(x) for x in bottleneck_values)
with open(bottleneck_path, 'w') as bottleneck_file:
  bottleneck_file.write(bottleneck_string)
       def get_or_create_bottleneck(sess, image_lists, label_name, index, image_dir,
                            category, bottleneck_dir, jpeg_data_tensor,
                            decoded_image_tensor, resized_input_tensor,
                            bottleneck_tensor, architecture):
label_lists = image_lists[label_name]
sub_dir = label_lists['dir']
sub_dir_path = os.path.join(bottleneck_dir, sub_dir)
ensure_dir_exists(sub_dir_path)
bottleneck_path = get_bottleneck_path(image_lists, label_name, index,
                                      bottleneck_dir, category, architecture)
if not os.path.exists(bottleneck_path):
  create_bottleneck_file(bottleneck_path, image_lists, label_name, index,
                         image_dir, category, sess, jpeg_data_tensor,
```

```
                    decoded_image_tensor, resized_input_tensor,
                    bottleneck_tensor)
  with open(bottleneck_path, 'r') as bottleneck_file:
   bottleneck_string = bottleneck_file.read()
  did_hit_error = False
  try:
   bottleneck_values = [float(x) for x in bottleneck_string.split(',')]
except ValueError:
   tf.logging.warning('Invalid float found, recreating bottleneck')
   did_hit_error = True
  if did_hit_error:
   create_bottleneck_file(bottleneck_path, image_lists, label_name, index,
                    image_dir, category, sess, jpeg_data_tensor,
                    decoded_image_tensor, resized_input_tensor,
                    bottleneck_tensor)
   with open(bottleneck_path, 'r') as bottleneck_file:
    bottleneck_string = bottleneck_file.read()
   bottleneck_values = [float(x) for x in bottleneck_string.split(',')]
  return bottleneck_values
   def cache_bottlenecks(sess, image_lists, image_dir, bottleneck_dir,
                jpeg_data_tensor, decoded_image_tensor,
                resized_input_tensor, bottleneck_tensor, architecture):
  how_many_bottlenecks = 0
  ensure_dir_exists(bottleneck_dir)
  for label_name, label_lists in image_lists.items():
```

```
  for category in ['training', 'testing', 'validation']:
    category_list = label_lists[category]
    for index, unused_base_name in enumerate(category_list):
      get_or_create_bottleneck(
          sess, image_lists, label_name, index, image_dir, category,
          bottleneck_dir, jpeg_data_tensor, decoded_image_tensor,
          resized_input_tensor, bottleneck_tensor, architecture)
how_many_bottlenecks += 1
      if how_many_bottlenecks % 100 == 0:
        tf.logging.info(
            str(how_many_bottlenecks) + ' bottleneck files created.')
      def get_random_cached_bottlenecks(sess, image_lists, how_many, category,
                                        bottleneck_dir, image_dir, jpeg_data_tensor,
                                        decoded_image_tensor, resized_input_tensor,
                                        bottleneck_tensor, architecture):
class_count = len(image_lists.keys())
bottlenecks = []
ground_truths = []
filenames = []
if how_many >= 0:
  for unused_i in range(how_many):
    label_index = random.randrange(class_count)
    label_name = list(image_lists.keys())[label_index]
    image_index = random.randrange(MAX_NUM_IMAGES_PER_CLASS + 1)
    image_name = get_image_path(image_lists, label_name, image_index,
```

```
                    image_dir, category)
    bottleneck = get_or_create_bottleneck(
        sess, image_lists, label_name, image_index, image_dir, category,
        bottleneck_dir, jpeg_data_tensor, decoded_image_tensor,
        resized_input_tensor, bottleneck_tensor, architecture)
    ground_truth = np.zeros(class_count, dtype=np.float32)
    ground_truth[label_index] = 1.0
    bottlenecks.append(bottleneck)
    ground_truths.append(ground_truth)
    filenames.append(image_name)
else:
  for label_index, label_name in enumerate(image_lists.keys()):
    for image_index, image_name in enumerate(
        image_lists[label_name][category]):
      image_name = get_image_path(image_lists, label_name, image_index,
                                  image_dir, category)
      bottleneck = get_or_create_bottleneck(
          sess, image_lists, label_name, image_index, image_dir, category,
          bottleneck_dir, jpeg_data_tensor, decoded_image_tensor,
          resized_input_tensor, bottleneck_tensor, architecture)
      ground_truth = np.zeros(class_count, dtype=np.float32)
      ground_truth[label_index] = 1.0
      bottlenecks.append(bottleneck)
      ground_truths.append(ground_truth)
      filenames.append(image_name)
```

```
return bottlenecks, ground_truths, filenames

    def get_random_distorted_bottlenecks(
  sess, image_lists, how_many, category, image_dir, input_jpeg_tensor,
  distorted_image, resized_input_tensor, bottleneck_tensor):
class_count = len(image_lists.keys())
bottlenecks = []
ground_truths = []
for unused_i in range(how_many):
  label_index = random.randrange(class_count)
  label_name = list(image_lists.keys())[label_index]
  image_index = random.randrange(MAX_NUM_IMAGES_PER_CLASS + 1)
  image_path = get_image_path(image_lists, label_name, image_index, image_dir,
                              category)
  if not gfile.Exists(image_path):
    tf.logging.fatal('File does not exist %s', image_path)
  jpeg_data = gfile.FastGFile(image_path, 'rb').read()
  distorted_image_data = sess.run(distorted_image,
                                  {input_jpeg_tensor: jpeg_data})
  bottleneck_values = sess.run(bottleneck_tensor,
                               {resized_input_tensor: distorted_image_data})
  bottleneck_values = np.squeeze(bottleneck_values)
  ground_truth = np.zeros(class_count, dtype=np.float32)
  ground_truth[label_index] = 1.0
  bottlenecks.append(bottleneck_values)
  ground_truths.append(ground_truth)
```

```
return bottlenecks, ground_truths
    def should_distort_images(flip_left_right, random_crop, random_scale,
                              random_brightness):
return (flip_left_right or (random_crop != 0) or (random_scale != 0) or
        (random_brightness != 0))
     def add_input_distortions(flip_left_right, random_crop, random_scale,
                              random_brightness, input_width, input_height,
                              input_depth, input_mean, input_std):
jpeg_data = tf.placeholder(tf.string, name='DistortJPGInput')
decoded_image = tf.image.decode_jpeg(jpeg_data, channels=input_depth)
decoded_image_as_float = tf.cast(decoded_image, dtype=tf.float32)
decoded_image_4d = tf.expand_dims(decoded_image_as_float, 0)
margin_scale = 1.0 + (random_crop / 100.0)
resize_scale = 1.0 + (random_scale / 100.0)
margin_scale_value = tf.constant(margin_scale)
resize_scale_value = tf.random_uniform(tensor_shape.scalar(),
                                       minval=1.0,
                                       maxval=resize_scale)
scale_value = tf.multiply(margin_scale_value, resize_scale_value)
precrop_width = tf.multiply(scale_value, input_width)
precrop_height = tf.multiply(scale_value, input_height)
precrop_shape = tf.stack([precrop_height, precrop_width])
precrop_shape_as_int = tf.cast(precrop_shape, dtype=tf.int32)
precropped_image = tf.image.resize_bilinear(decoded_image_4d,
                                            precrop_shape_as_int)
```

```
  precropped_image_3d = tf.squeeze(precropped_image, squeeze_dims=[0])
  cropped_image = tf.random_crop(precropped_image_3d,
                                 [input_height, input_width, input_depth])
  if flip_left_right:
    flipped_image = tf.image.random_flip_left_right(cropped_image)
  else:
    flipped_image = cropped_image
  brightness_min = 1.0 - (random_brightness / 100.0)
  brightness_max = 1.0 + (random_brightness / 100.0)
  brightness_value = tf.random_uniform(tensor_shape.scalar(),
                                       minval=brightness_min,
                                       maxval=brightness_max)
  brightened_image = tf.multiply(flipped_image, brightness_value)
  offset_image = tf.subtract(brightened_image, input_mean)
  mul_image = tf.multiply(offset_image, 1.0 / input_std)
  distort_result = tf.expand_dims(mul_image, 0, name='DistortResult')
  return jpeg_data, distort_result
def variable_summaries(var):
  with tf.name_scope('summaries'):
    mean = tf.reduce_mean(var)
    tf.summary.scalar('mean', mean)
    with tf.name_scope('stddev'):
      stddev = tf.sqrt(tf.reduce_mean(tf.square(var - mean)))
    tf.summary.scalar('stddev', stddev)
    tf.summary.scalar('max', tf.reduce_max(var))
```

```
    tf.summary.scalar('min', tf.reduce_min(var))
    tf.summary.histogram('histogram', var)
      def add_final_training_ops(class_count, final_tensor_name, bottleneck_tensor,
                        bottleneck_tensor_size):
  with tf.name_scope('input'):
    bottleneck_input = tf.placeholder_with_default(
        but found '%s' for architecture '%s'""",
         size_string, architecture)
      return None
    if len(parts) == 3:
      is_quantized = False
    else:
      if parts[3] != 'quantized':
        tf.logging.error(
"Couldn't understand architecture suffix '%s' for '%s'", parts[3],
            architecture)
        return None
      is_quantized = True
    data_url = 'http://download.tensorflow.org/models/mobilenet_v1_'
    data_url += version_string + '_' + size_string + '_frozen.tgz'
    bottleneck_tensor_name = 'MobilenetV1/Predictions/Reshape:0'
    bottleneck_tensor_size = 1001
    input_width = int(size_string)
    input_height = int(size_string)
    input_depth = 3
```

```
    resized_input_tensor_name = 'input:0'
    if is_quantized:
      model_base_name = 'quantized_graph.pb'
    else:
      model_base_name = 'frozen_graph.pb'
    model_dir_name = 'mobilenet_v1_' + version_string + '_' + size_string
    model_file_name = os.path.join(model_dir_name, model_base_name)
    input_mean = 127.5
    input_std = 127.5
  else:
    tf.logging.error("Couldn't understand architecture name '%s'", architecture)
    raise ValueError('Unknown architecture', architecture)
return {
      'data_url': data_url,
      'bottleneck_tensor_name': bottleneck_tensor_name,
      'bottleneck_tensor_size': bottleneck_tensor_size,
      'input_width': input_width,
      'input_height': input_height,
      'input_depth': input_depth,
      'resized_input_tensor_name': resized_input_tensor_name,
      'model_file_name': model_file_name,
      'input_mean': input_mean,
      'input_std': input_std,
  }
def add_jpeg_decoding(input_width, input_height, input_depth, input_mean,
```

```
                input_std):
  jpeg_data = tf.placeholder(tf.string, name='DecodeJPGInput')
  decoded_image = tf.image.decode_jpeg(jpeg_data, channels=input_depth)
  decoded_image_as_float = tf.cast(decoded_image, dtype=tf.float32)
  decoded_image_4d = tf.expand_dims(decoded_image_as_float, 0)
  resize_shape = tf.stack([input_height, input_width])
  resize_shape_as_int = tf.cast(resize_shape, dtype=tf.int32)
  resized_image = tf.image.resize_bilinear(decoded_image_4d,
                                           resize_shape_as_int)
  offset_image = tf.subtract(resized_image, input_mean)
  mul_image = tf.multiply(offset_image, 1.0 / input_std)
  return jpeg_data, mul_image
def main(_):
  # Needed to make sure the logging output is visible.
  # See https://github.com/tensorflow/tensorflow/issues/3047
  tf.logging.set_verbosity(tf.logging.INFO)
 # Prepare necessary directories  that can be used during training
  prepare_file_system()
 # Gather information about the model architecture we'll be using.
  model_info = create_model_info(FLAGS.architecture)
  if not model_info:
    tf.logging.error('Did not recognize architecture flag')
    return -1
  # Set up the pre-trained graph.
  maybe_download_and_extract(model_info['data_url'])
```

```
  graph, bottleneck_tensor, resized_image_tensor = (
      create_model_graph(model_info))
       # Look at the folder structure, and create lists of all the images.
  image_lists = create_image_lists(FLAGS.image_dir, FLAGS.testing_percentage,
                                   FLAGS.validation_percentage)
  class_count = len(image_lists.keys())
  if class_count == 0:
    tf.logging.error('No valid folders of images found at ' + FLAGS.image_dir)
    return -1
  if class_count == 1:
    tf.logging.error('Only one valid folder of images found at ' +
                     FLAGS.image_dir +
                     ' - multiple classes are needed for classification.')
    return -1
  # See if the command-line flags mean we're applying any distortions.
  do_distort_images = should_distort_images(
      FLAGS.flip_left_right, FLAGS.random_crop, FLAGS.random_scale,
      FLAGS.random_brightness)
with tf.Session(graph=graph) as sess:
    # Set up the image decoding sub-graph.
    jpeg_data_tensor, decoded_image_tensor = add_jpeg_decoding(
        model_info['input_width'], model_info['input_height'],
        model_info['input_depth'], model_info['input_mean'],
        model_info['input_std'])
```

```
if do_distort_images:
  # We will be applying distortions, so setup the operations we'll need.
  (distorted_jpeg_data_tensor,
   distorted_image_tensor) = add_input_distortions(
       FLAGS.flip_left_right, FLAGS.random_crop, FLAGS.random_scale,
       FLAGS.random_brightness, model_info['input_width'],
       model_info['input_height'], model_info['input_depth'],
       model_info['input_mean'], model_info['input_std'])
else:
  # We'll make sure we've calculated the 'bottleneck' image summaries and
  # cached them on disk.
  cache_bottlenecks(sess, image_lists, FLAGS.image_dir,
                    FLAGS.bottleneck_dir, jpeg_data_tensor,
                    decoded_image_tensor, resized_image_tensor,
                    bottleneck_tensor, FLAGS.architecture)
# Add the new layer that we'll be training.
(train_step, cross_entropy, bottleneck_input, ground_truth_input,
 final_tensor) = add_final_training_ops(
     len(image_lists.keys()), FLAGS.final_tensor_name, bottleneck_tensor,
     model_info['bottleneck_tensor_size'])
# Create the operations we need to evaluate the accuracy of our new layer.
evaluation_step, prediction = add_evaluation_step(
    final_tensor, ground_truth_input)


# Merge all the summaries and write them out to the summaries_dir
```

```
merged = tf.summary.merge_all()
train_writer = tf.summary.FileWriter(FLAGS.summaries_dir + '/train',
                                     sess.graph)
  tf.logging.info('Save intermediate result to : ' +
                  intermediate_file_name)
  save_graph_to_file(sess, graph, intermediate_file_name)
# We've completed all our training, so run a final test evaluation on
# some new images we haven't used before.
test_bottlenecks, test_ground_truth, test_filenames = (
  get_random_cached_bottlenecks(
      sess, image_lists, FLAGS.test_batch_size, 'testing',
      FLAGS.bottleneck_dir, FLAGS.image_dir, jpeg_data_tensor,
      decoded_image_tensor, resized_image_tensor, bottleneck_tensor,
      FLAGS.architecture))
test_accuracy, predictions = sess.run(
  [evaluation_step, prediction],
  feed_dict={bottleneck_input: test_bottlenecks,
             ground_truth_input: test_ground_truth})
tf.logging.info('Final test accuracy = %.1f%% (N=%d)' %
                (test_accuracy * 100, len(test_bottlenecks)))
if FLAGS.print_misclassified_test_images:
 tf.logging.info('=== MISCLASSIFIED TEST IMAGES ===')
 for i, test_filename in enumerate(test_filenames):
  if predictions[i] != test_ground_truth[i].argmax():
   tf.logging.info('%70s  %s' %
```

```
                    (test_filename,
                     list(image_lists.keys())[predictions[i]]))
    # Write out the trained graph and labels with the weights stored as
    # constants.
    save_graph_to_file(sess, graph, FLAGS.output_graph)
    with gfile.FastGFile(FLAGS.output_labels, 'w') as f:
      f.write('\n'.join(image_lists.keys()) + '\n')
if __name__ == '__main__':
  parser = argparse.ArgumentParser()
  parser.add_argument(
      '--image_dir',
      type=str,
      default='',
      help='Path to folders of labeled images.')
  parser.add_argument(
      '--output_graph',
      type=str,
      default='/tmp/output_graph.pb',
      help='Where to save the trained graph.')
  parser.add_argument(
      '--intermediate_output_graphs_dir',
      type=str,
      default='/tmp/intermediate_graph/',
help='Where to save the intermediate graphs.')
  parser.add_argument(
```

```
    '--intermediate_store_frequency',
    type=int,
    default=0,
    help="""\
      How many steps to store intermediate graph. If "0" then will not
      store.\""")
  parser.add_argument(
    '--output_labels',
    type=str,
    default='/tmp/output_labels.txt',
    help='Where to save the trained graph\'s labels.')
  parser.add_argument(
    '--summaries_dir',
    type=str,
    default='/tmp/retrain_logs',
    help='Where to save summary logs for TensorBoard.')
  parser.add_argument(
    '--how_many_training_steps',
    type=int,
    default=4000,
    help='How many training steps to run before ending.')
  parser.add_argument(
    '--learning_rate',
    type=float,
    default=0.01,
```

```
    help='How large a learning rate to use when training.')
  parser.add_argument(
    '--testing_percentage',
    type=int,
    default=10,
    help='What percentage of images to use as a test set.')
  parser.add_argument(
    '--validation_percentage',
    type=int,
    default=10,
    help='What percentage of images to use as a validation set.')
  parser.add_argument(
    '--eval_step_interval',
    type=int,
    default=10,
    help='How often to evaluate the training results.')
  parser.add_argument(
    '--train_batch_size',
    type=int,
    default=100,
    help='How many images to train on at a time.')
  parser.add_argument(
    '--test_batch_size',
    type=int,
    default=-1,
```

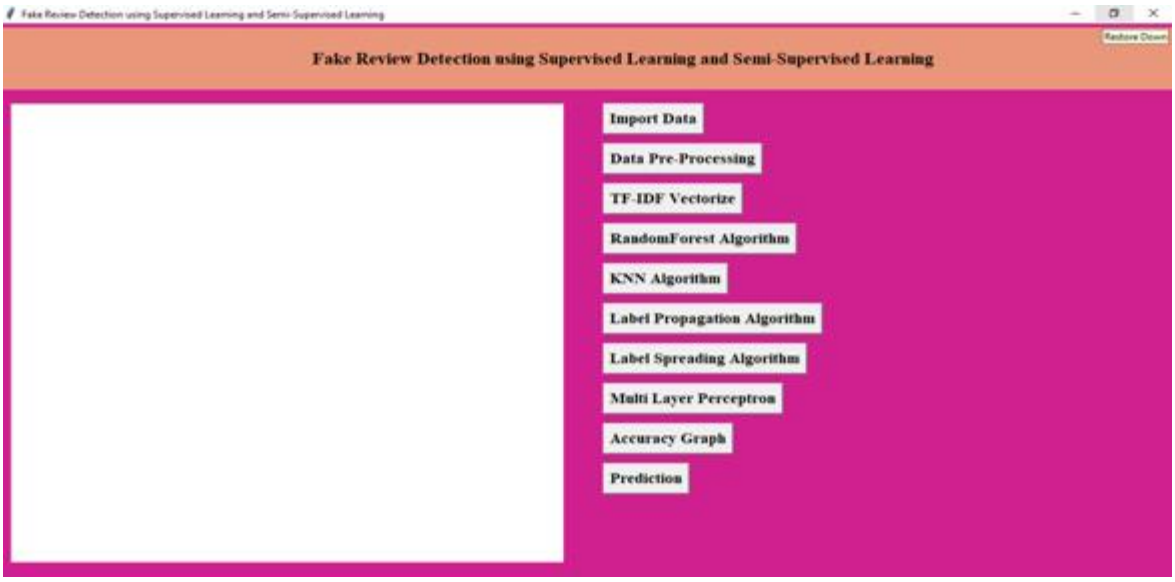
```
      help="""\
      How many images to test on. This test set is only used once, to evaluate
      the final accuracy of the model after training completes.
      A value of -1 causes the entire test set to be used, which leads to more
      stable results across runs.\""")
  parser.add_argument(
      '--validation_batch_size',
      type=int,
      default=100,
      help="""\
      How many images to use in an evaluation batch. This validation set is
      used much more often than the test set, and is an early indicator of how
      accurate the model is during training.
      A value of -1 causes the entire validation set to be used, which leads to
      more stable results across training iterations, but may be slower on large
      training sets.\
""")
  parser.add_argument(
      '--print_misclassified_test_images',
      default=False,
      help="""\
      Whether to print out a list of all misclassified test images.\
""",
      action='store_true'  )
  parser.add_argument(
```

```
      '--model_dir',
      type=str,
      default='/tmp/imagenet',
      help="""\
      Path to classify_image_graph_def.pb,
      imagenet_synset_to_human_label_map.txt, and
      imagenet_2012_challenge_label_map_proto.pbtxt.\
"""")
  parser.add_argument(
      '--bottleneck_dir',
      type=str,
      default='/tmp/bottleneck',
      help='Path to cache bottleneck layer values as files.')
  parser.add_argument(
      '--final_tensor_name',
      type=str,
      default='final_result',
      help="""\
      The name of the output classification layer in the retrained graph.\
"""")
  parser.add_argument(
      '--flip_left_right',
      default=False,
      help="""\
      Whether to randomly flip half of the training images horizontally.\""",
```

```
    action='store_true')
parser.add_argument(
    '--random_crop',
    type=int,
    default=0,
    help="""\
    A percentage determining how much of a margin to randomly crop off the
    training images.\ """)
parser.add_argument(
    '--random_scale',
    type=int,
    default=0,
    help="""\
    A percentage determining how much to randomly scale up the size of the
    training images by.\ """)
parser.add_argument(
    '--random_brightness',
    type=int,
    default=0,
    help="""\
    A percentage determining how much to randomly multiply the training image
    input pixels up or down by.\""")
parser.add_argument(
    '--architecture',
```

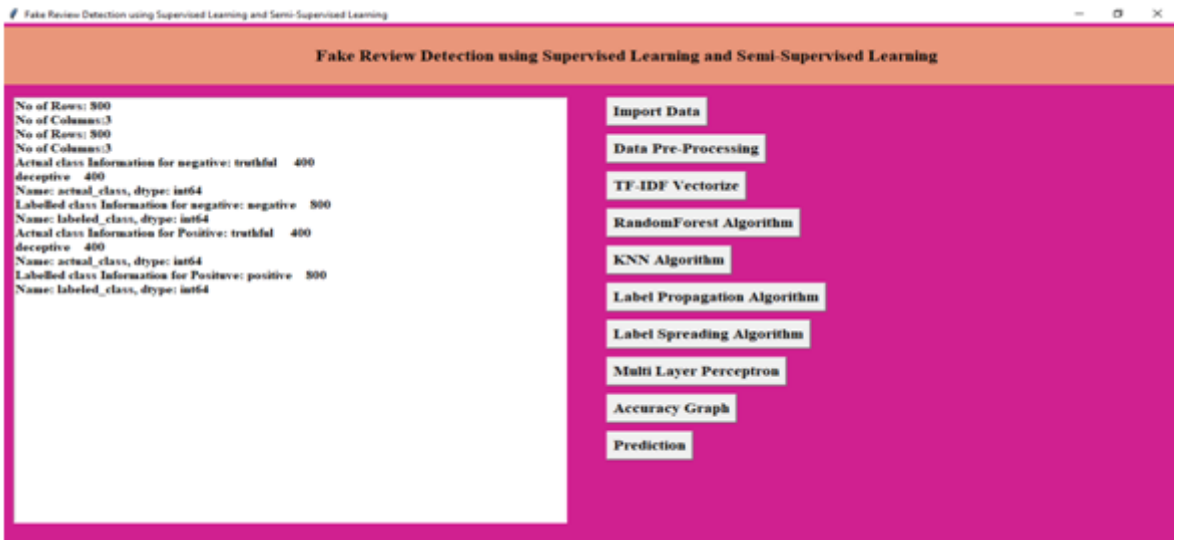
6. OUTPUT SCREENS

- Open Anaconda Prompt.
- Goto Project Folder Path usind cd command Run python frd.py



6.1.1.1 Home page

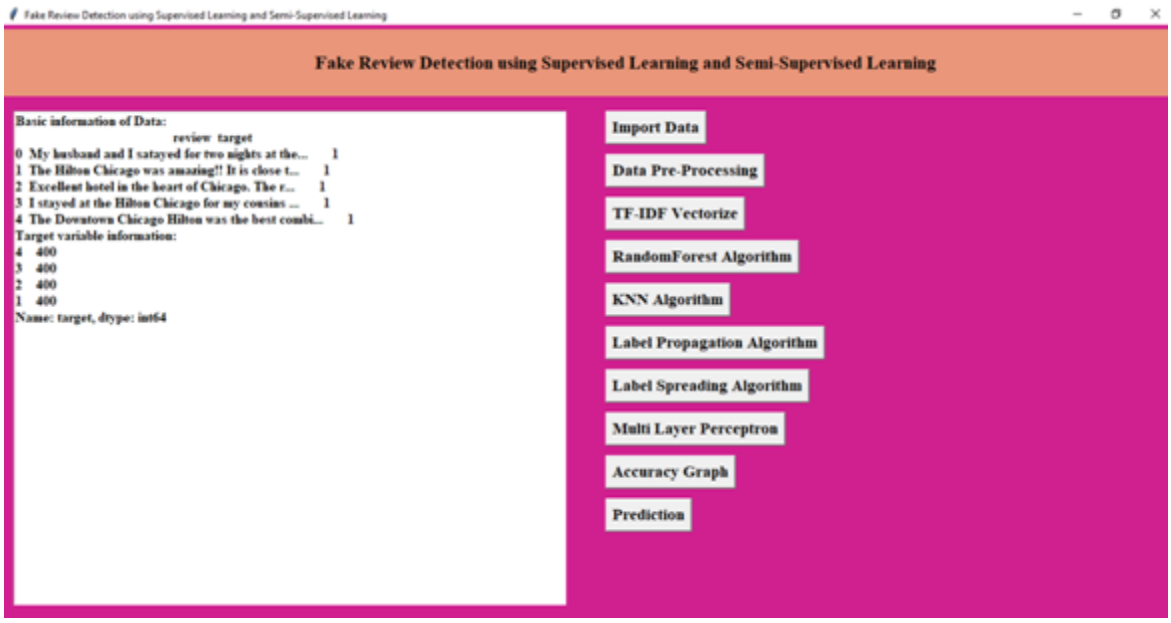
- Above Desktop application is opened and click on Import Data



6.1.1.2 Click on import data

- Data is loaded into from the “Positive review” and “Negative Review” folder and it will print basic information of data.

- Now click on Preprocess Button



6.1.1.3 click on Data Pre-processing Button

- Data frame is created and we are converting target variable into four classes as below.
- Highly positive (1) : The review is marked as positive and it is deceptive .
- Positive (2) : A review is marked as positive and it is truly positive .
- Negative (3) : A review is marked and negative and it is truly negative .
- Highly Negative (4) : A review is marked and negative and it is deceptive .
- Now click on “TFIDF Vectorize”

6.1.1.4 click on “TFIDF Vectorize

6.1.1.5 Feature Extraction using TFIDF

➢ word_tokenize converts each review into lowercase and append each character of review into a list

➢ the we tag parts of speech of each word and lemmatize the words(reduce the word to root as in dictionary format)

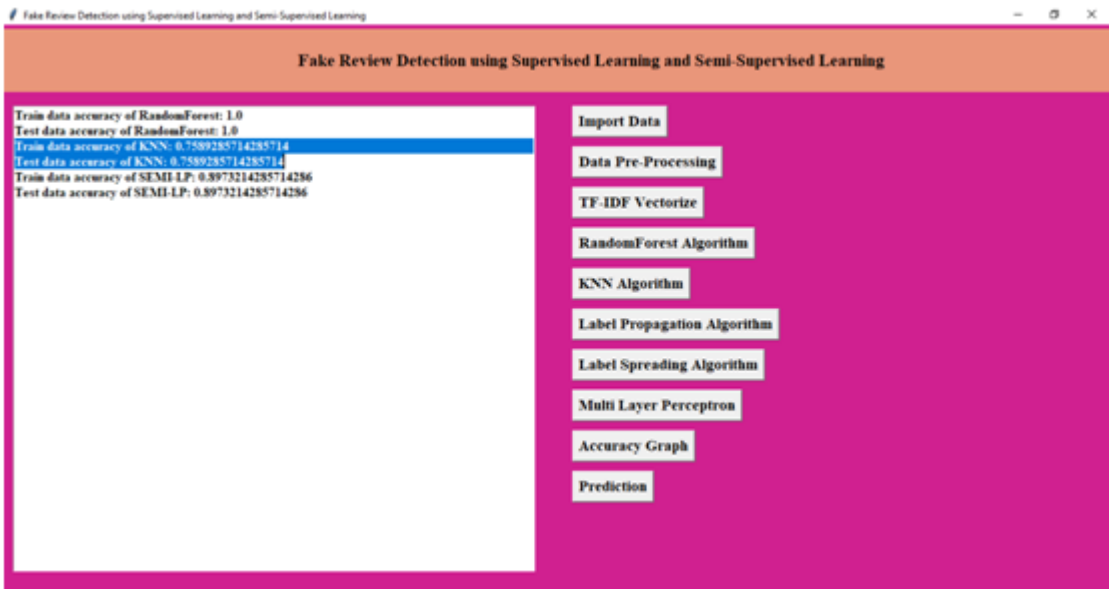
➢ apped these lists to a column names reviews_tokenized

- Feature Extraction using TFIDF., Now Click on “Random Forest Algoritm”

6.1.1.6 Click on “Random Forest Algoritm

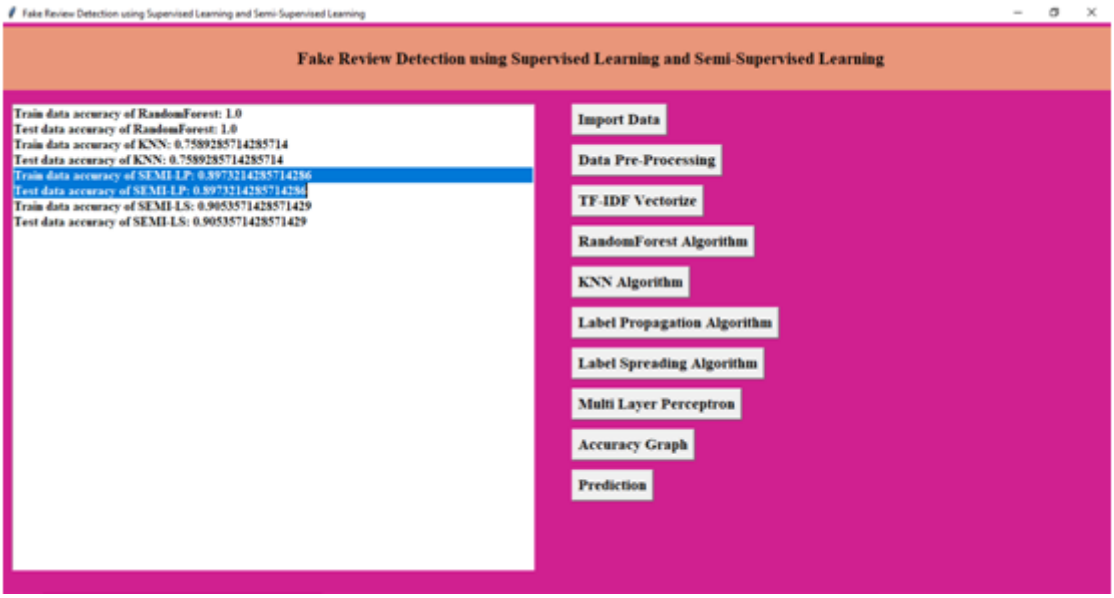
6.1.1.7 display train and test data accuracy

- Train data accuracy of KNN: 0.7589285714285714
- Test data accuracy of KNN: 0.7589285714285714
- Now Click on “Label Propagation”.



6.1.1.8 Click on label propagation

- Train data accuracy of SEMI-LP: 0.8973214285714286
- Test data accuracy of SEMI-LP: 0.8973214285714286
- Now click on “Label Spreading Algorithm”



6.1.1.9 click on “Label Spreading Algorithm”

- Train data accuracy of SEMI-LS: 0.9053571428571429
- Test data accuracy of SEMI-LS: 0.9053571428571429

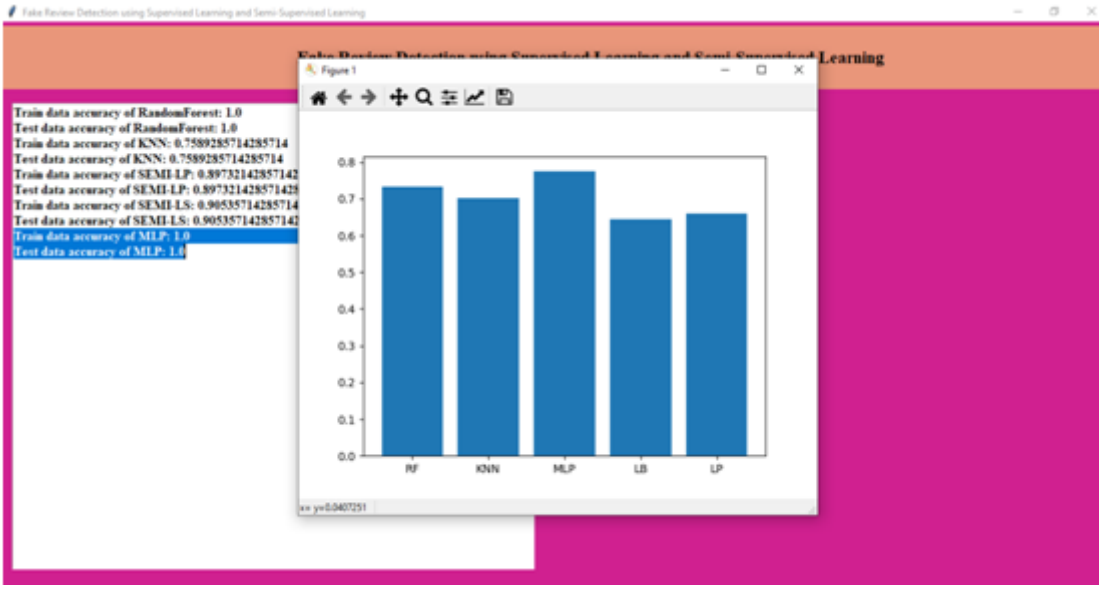
- Now Click on “Multi Layer Perceptron”

6.1.1.10 Click on “Multi Layer Perceptron”

- Train data accuracy of MLP: 1.0

- Test data accuracy of MLP: 1.0

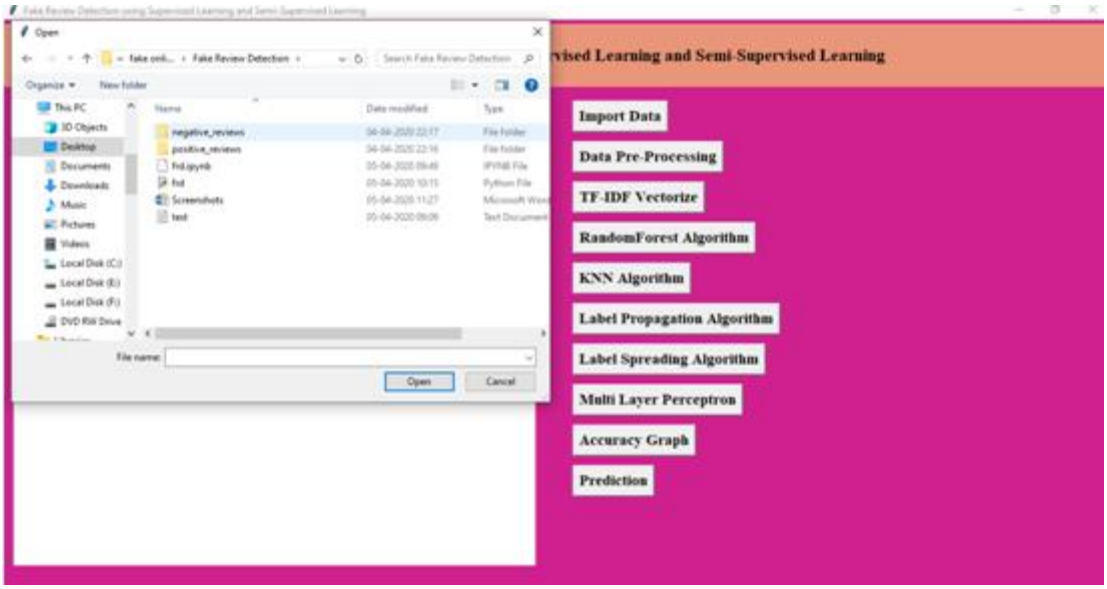
- Now Click on “Graph” button which will display accuracy comparision.



6.1.1.11 display accuracy comparision

- Now click on Prediction button.

- Need to upload the test file with review.



6.1.1.12 click on Prediction button

6.1.1.13 Shows Prediction is Fake Review (Negative)

- For given data, Prediction is Fake Review (Negative)

7. TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. Software system meets its requirements and user expectations and does not fail in an unacceptable manner.

TYPES OF TESTS

Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs.

Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields.

Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail

Test objectives

All field entries must work properly.

Pages must be activated from the identified link.

The entry screen, messages and responses must not be delayed.

Features to be tested

Verify that the entries are of the correct format

No duplicate entries should be allowed

All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

8. CONCLUSION

We have shown several semi-supervised and supervised text mining techniques for detecting fake online reviews in this research. We have combined features from several research works to create a better feature set. Also we have tried some other classifer that were not used on the previous work. Thus, we have been able to increase the accuracy of previous semi- supervised techniques done by Jiten et al. [8]. We have also found out that supervised Naive Bayes classifier gives the highest accuracy. This ensures that our dataset is labeled well In our research work we have worked on just user reviews. In future, user behaviors can be combined with texts to construct a better model for classification. Advanced prepro- cessing tools for tokenization can be used to make the dataset more precise. Evaluation of the effectiveness of the proposed methodology can be done for a larger data set. This research work is being done only for English reviews. It can be done for Bangla and several other languages.

8.1 Future Enhancement

In our research work we have worked on just user reviews. In future, user behaviors can be combined with texts to construct a better model for classification. Advanced prepro- cessing tools for tokenization can be used to make the dataset more precise. Evaluation of the effectiveness of the proposed methodology can be done for a larger data set. This research work is being done only for English reviews. It can be done for Bangla and several other languages.

9. REFERENCES

Chengai Sun, Qiaolin Du, Gang Tian, "Exploiting Product Related Review Features for Fake Review Detection" in Mathematical Problems in Engineering, 2016.

A. Heydari, M. A. Tavakoli, N. Salim, Z. Heydari, "Detection of review spam: a survey", Expert Systems with Applications, vol. 42, no. 7, pp. 3634-3642, 2015.

M. Ott, Y. Choi, C. Cardie, J. T. Hancock, "Finding deceptive opinion spam by any stretch of the imagination", Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (ACL-HLT), vol. 1, pp. 309-319, June 2011.

J. W. Pennebaker, M. E. Francis, R. J. Booth, Linguistic Inquiry and Word Count: Liwc, vol. 71, 2001

S. Feng, R. Banerjee, Y. Choi, "Syntactic stylometry for deception detection", Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics: Short Papers, vol. 2, 2012.

J. Li, M. Ott, C. Cardie, E. Hovy, "Towards a general rule for identifying deceptive opinion spam", Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (ACL), 2014

E. P. Lim, V.-A. Nguyen, N. Jindal, B. Liu, H. W. Lauw, "Detecting product review spammers using rating behaviors", Proceedings of the 19th ACM International Conference on Information and Knowledge Management (CIKM), 2010.

K. Rout, A. Dalmia, K.-K. R. Choo, "Revisiting semi-supervised learning for online deceptive review detection", IEEE Access, vol. 5, pp. 1319-1327, 2017.

J. Karimpour, A. A. Noroozi, S. Alizadeh, "Web spam detection by learning from small labeled samples", International Journal of Computer Applications, vol. 50, no. 21, pp. 1-5, July 2012.